

OOP Fundamentals

Note: Your source material for OOP is thin (mostly bare code screenshots with little explanation), so this topic — and most of the OOPS drafts — are built substantially from general knowledge, using your material's examples as anchors where they exist. C++ and Java snippets are shown side by side throughout, since your materials mix both.

1. What Problem Does OOP Actually Solve?

Before OOP, large programs were organized around **procedures acting on data** — functions and the data they operate on were kept separate, which meant:

- Data could be modified by *any* function, anywhere, with no way to guarantee it stayed in a valid state.
- Modeling real-world entities (a car, a bank account, an employee) as a scattered set of variables and free functions didn't match how people actually think about the problem.
- Code reuse meant copy-pasting or awkward function composition — there was no natural way to say "this new thing is basically that other thing, plus a bit more."

OOP's core idea: bundle data and the functions that operate on it together into a single unit — an **object** — and organize programs as a collection of interacting objects, each responsible for its own data and behavior.

2. Classes vs. Objects

- **Class** — a **blueprint/template** that defines what data (fields/attributes) and behavior (methods) its instances will have. No memory is allocated for actual data until an object is created.
- **Object** — a concrete **instance** of a class, with its own actual storage for the class's fields.

```
// C++
class Cat {
public:
    string name, colour, toy;
    Cat() { name = "Unknown"; colour = "Unknown"; toy = "Unknown"; }
};

int main() {
    Cat cat1;                // cat1 is an OBJECT of class Cat
    cout << "Cat 1 name: " << cat1.name;
}
```

```
// Java
class Cat {
    String name, colour, toy;
    Cat() { name = "Unknown"; colour = "Unknown"; toy = "Unknown"; }
}

public class Main {
    public static void main(String[] args) {
        Cat cat1 = new Cat();    // cat1 is an OBJECT of class Cat
        System.out.println("Cat 1 name: " + cat1.name);
    }
}
```

Interview soundbite: "A class is a type; an object is a value of that type. The same relationship as `int` (the type) and `5` (a value) — just for a programmer-defined, structured type that bundles data with behavior."

3. The Four Pillars of OOP

Pillar	One-line definition
Encapsulation	Bundling data and the methods that operate on it together, and hiding the data from direct outside access/interference
Abstraction	Hiding implementation details , exposing only the functionality — the user knows <i>what</i> a method does, not <i>how</i>
Inheritance	One class acquiring the properties/methods of another, enabling hierarchical classification and reuse
Polymorphism	The same interface/method call behaving differently depending on the actual underlying object/type

A running example — a car company, `K Motors`, building both petrol and electric cars:

- **Encapsulation** — `price` is stored as a `private` field; outside code can only interact with it through `getPrice()` / `setPrice()`, never touching it directly.
- **Abstraction** — calling `drive()` doesn't reveal (or require the caller to know) whether the car underneath is a combustion engine or an electric motor — that complexity is hidden behind one simple method call.
- **Inheritance** — `ElectricCar` **extends** `Car`, automatically gaining all of `Car`'s fields and methods, adding only what's specific to being electric.
- **Polymorphism** — calling `fuelUp()` fills a petrol tank for one car and charges a battery for another, **through the exact same method call** — the caller doesn't need to know or care which one it actually is.

Why these four, specifically? Each pillar solves a distinct structural problem: encapsulation protects data integrity, abstraction manages complexity for the *caller*, inheritance enables reuse and hierarchical modeling, and polymorphism lets code written against a general interface work correctly for *any* specific implementation of it, present or future. They're independent axes — a language or design can have some without others (see §4).

Encapsulation in practice

```
// C++
class Car {
    double price;           // private by default in a class
public:
    double getPrice() { return price; }
    void setPrice(double p) { if (p >= 0) price = p; } // can validate before allowing a change
};
```

```
// Java
class Car {
    private double price;
    public double getPrice() { return price; }
    public void setPrice(double price) { if (price >= 0) this.price = price; }
}
```

Why bother with getters/setters instead of a public field? A public field can be set to *anything* by any calling code, with zero validation. A setter is a **controlled gate** — it can reject invalid values, trigger side effects (logging, recalculating a derived field), or later be changed internally without breaking any code that calls it (this is the seed of the *why* behind data hiding, developed further in the Access Modifiers & Encapsulation topic).

4. Object-Based vs. Object-Oriented Programming — a genuinely useful distinction

Object-based language: supports objects and encapsulation, but **not** inheritance or polymorphism (e.g., early Visual Basic, or using structs-with-functions in a language with no class hierarchy support).

Object-oriented language: supports **all four** pillars, including inheritance and polymorphism (Java, C++, Python, ...).

Why this distinction matters in an interview: it's a common trick question — "is JavaScript/early VB/some other language OOP?" The precise answer depends on whether it supports **inheritance and polymorphism**, not just "does it have objects." Having objects and hiding data (encapsulation) alone only gets you to *object-based*.

5. Why This Matters for Software Design (not just syntax)

- **Encapsulation** → protects **invariants** (rules that must always hold, e.g., "balance can never go negative") by controlling the only paths through which state can change.
- **Abstraction** → lets a caller depend on a **stable, simple interface** while the implementation underneath is free to change (optimize, refactor, even swap algorithms entirely) without breaking anything that calls it.
- **Inheritance** → models **"is-a" relationships** and enables code reuse — but (developed in the Composition vs Inheritance topic) it's not always the right tool, and overusing it is a very common real-world design mistake.
- **Polymorphism** → is what makes **extensible** systems possible: new types can be added later that work seamlessly with existing code written against a shared interface, without that existing code needing to change at all (this is directly connected to the Open/Closed Principle in the SOLID topic).

Interview soundbite: "The four pillars aren't academic trivia — each one is a direct answer to a specific, recurring software engineering pain point: uncontrolled state mutation, leaky implementation details, code duplication across similar types, and rigid code that has to be rewritten every time a new case shows up."

Interview Questions With Answers

Q1. What's the precise difference between a class and an object?

Answer: A class is a blueprint/type definition — it specifies what fields and methods its instances will have, but has no actual data storage allocated for it by itself. An object is a concrete instance of a class, with its own real memory allocated for each of the class's fields — you can create many objects from the same class, each with independent state.

Q2. Define all four pillars of OOP in one sentence each, and give a concrete example of each from a single running scenario.

Answer: Using a car company as the scenario: Encapsulation is keeping a car's `price` private and only accessible via `getPrice()` / `setPrice()`. Abstraction is calling `drive()` without needing to know whether the underlying engine is petrol or electric. Inheritance is `ElectricCar` extending `Car` to reuse its fields and methods while adding electric-specific behavior. Polymorphism is calling the same `fuelUp()` method on any car object and having it correctly fill a tank or charge a battery depending on the actual object's type.

Q3. What's the difference between an object-based language and an object-oriented language?

Answer: An object-based language supports objects and encapsulation but lacks support for inheritance and polymorphism (early Visual Basic is the classic example) — you can bundle data and methods together, but you can't build a class hierarchy or have one interface behave differently for different underlying types. An object-oriented language supports all four pillars, including inheritance and polymorphism, allowing genuine hierarchical modeling and interface-based extensibility.

Q4. Why is exposing a public field considered worse practice than exposing a private field with a getter/setter, even if the setter currently does nothing but assign the value?

Answer: Even a currently-trivial setter creates a controlled boundary — a single place through which all external modification of that field must pass. This means validation, logging, or side-effect logic can be added later without changing the public interface (no calling code needs to be rewritten), and it means the internal representation of the field could even change entirely (e.g., splitting one field into two, or computing it lazily) without breaking external callers, since they were never depending on direct field access in the first place. A public field offers none of this — any code anywhere can set it to any value, with no gate at all, and changing its internal representation later would break every direct external reference to it.

Q5. Why does abstraction matter for a caller, distinct from why encapsulation matters?

Answer: Encapsulation is primarily about protecting an object's *own* internal state from being corrupted by external code. Abstraction is about what a *caller* needs to know to use something correctly — it lets the caller depend on a simple, stable description of *what* an operation does, without needing to understand or depend on *how* it's implemented internally. This distinction matters because abstraction is what allows an implementation to be completely rewritten or optimized later (e.g., swapping how `drive()` works internally) without requiring any change to code that merely calls `drive()` — the caller's mental model and code never depended on those internal details in the first place.

Q6. Why is inheritance grouped with polymorphism as "two of the four pillars," rather than inheritance alone being considered sufficient for code reuse?

Answer: Inheritance by itself only gives you structural reuse — a subclass automatically has the same fields and methods as its parent, reducing duplicated code. But without polymorphism, calling code would still need to know the *exact* concrete type of an object to call the right version of a method, defeating much of inheritance's practical value. Polymorphism is what lets a single piece of calling code work correctly across an entire family of related types via one shared interface/method signature — inheritance provides the family relationship, polymorphism provides the ability to treat the whole family uniformly through that relationship, and the two are typically used together precisely because each solves half of the same underlying reuse-and-extensibility problem.

Q7. Scenario: A junior developer argues that since their language technically has classes and lets them create objects, their codebase is "fully object-oriented" — even though every class is a flat data container with public fields, no class ever extends another, and there are no virtual/overridable methods anywhere. Evaluate this claim using the concepts from this topic.

Answer: The claim is incorrect by the standard definitions used here. Having classes and creating objects with public fields demonstrates, at best, the *object* part of object-based programming — and without any data-hiding at all (public fields, no getters/validation), it doesn't even clearly demonstrate encapsulation in a meaningful sense. Critically, the codebase has no inheritance (no class ever extends another) and no polymorphism (no virtual/overridable methods, so there's no possibility of one interface behaving differently for different types) — by definition, a codebase lacking both of these is not object-oriented, regardless of whether the underlying language is capable of supporting OOP. This codebase would more accurately be described as using a language capable of OOP, while itself being written in a purely object-based (at best) or even procedural style with data grouped into structs.